

The Architecture Review Checklist: 41 Questions I Ask

The questions that come up in almost every cloud architecture review I run — compute, ownership, security, data, networking, resilience, operations and cost — as a checklist you can use before yours.

September 26, 2026 · 4 min read

<https://brianpfeil.com/architecture/architecture-review-checklist/>

I've sat on the reviewing side of a few hundred cloud architecture reviews. Different teams, vendors and business problems — but the same forty-odd questions come up again and again. Here they are, grouped the way I think about them.

Use it two ways: **if you're presenting**, answer every question before the meeting and the review becomes a conversation instead of an interrogation. **If you're reviewing**, it's a starting point; add the questions your organisation keeps having to ask.

The goal of a review isn't to catch a team out. It's to make sure the architecture that gets **built** is the one everyone **agreed** to, and that nobody is surprised six months later by a bill, an outage or an audit.

1. Compute: why not something more managed?

My default order of preference is SaaS, then serverless, then containers, then servers ([more on that ladder](#)). Most of these questions are about moving one rung up.

- ☐ **Could this be SaaS?** If not, write down why not.
- ☐ **Why not serverless?** The burden of proof is on the less-managed option.
- ☐ **If it needs containers, why not a serverless container platform** rather than a self-managed cluster?
- ☐ **Is the workload really stateless**, or does it quietly depend on local disk?
- ☐ **Is this a lift-and-shift?** Where's the chance to move up the stack later?
- ☐ **What starts each process?** Could a cron job or a poller become an event?

- ☐ **Will anything hit a limit?** Function timeouts, payload sizes, header sizes, API quotas. Check the service quotas *before* you design around a service.
-

2. Vendor and ownership

Vendor products show up in a lot of reviews, and the questions are always about who does the work after go-live.

- ☐ **Who installs, patches and upgrades it,** and on what cadence?
 - ☐ **How does the vendor ship updates?** Are their images and packages scanned?
 - ☐ **Who supports it in production,** and what does that support actually cover?
 - ☐ **What's the exact split of responsibility** between you, the vendor and the cloud provider?
-

3. Identity, access and secrets

- ☐ **How do people sign in?** Does it support single sign-on over SAML or OIDC?
 - ☐ **For each component: what authenticates to it?** If it isn't native cloud IAM, why not?
 - ☐ **Where do credentials, keys and tokens live,** who can read them, and is rotation automatic?
 - ☐ **Is any automation running on a person's credentials?** That's an anti-pattern — use a role.
 - ☐ **Does every function and task have its own least-privilege role?** Prefer narrow custom policies over broad managed ones.
 - ☐ **For a public endpoint with no auth, what stops abuse?** A web application firewall, throttling, API keys, all three?
-

4. Data and compliance

- ☐ **What's the data classification and business criticality?** Is any of it regulated?
 - ☐ **How is data encrypted at rest and in transit?** Does anything need encrypting on the client side?
 - ☐ **How long is data kept?** Can it move to cheaper storage tiers, or be deleted?
-

- ☐ **Are there audit requirements**, and could you produce the evidence tomorrow?
- ☐ **Is sensitive data kept out of the logs?**
- ☐ **Does it matter where the data physically lives?**

5. Networking

- ☐ **Which ports and protocols does it need?** Anything inbound other than 443 needs a reason.
- ☐ **What has to be public, and why?**
- ☐ **How does it connect to other environments, on-premises and partners?** Who provides and who consumes each private link?
- ☐ **How is DNS handled?** Put a stable name in front of generated endpoints.
- ☐ **Why does the application care about IP addresses at all?** Identity beats IP allow-lists.

6. Resilience, high availability and disaster recovery

- ☐ **What are the recovery time and recovery point objectives?** Is DR even required?
- ☐ **Is it multi-AZ?** For anything that isn't, what compensates?
- ☐ **Is anything stateful sitting in the recovery path?**
- ☐ **Can lost data be rebuilt or replayed** from another source?
- ☐ **Are handlers idempotent**, so a retry is safe? Where do failures go — a dead-letter queue — and who replays them?

7. Observability and operations

- ☐ **Which logs and metrics exist, and which ones alarm?** Every error metric should either page someone or trigger self-healing. An error nobody hears about isn't monitored.
- ☐ **Are logs structured**, with metrics derived from them?
- ☐ **Is there distributed tracing** across the serverless pieces?

- ☐ Can it be built, deployed and run without anyone clicking in the console?

8. Delivery and cost

- ☐ Is all of the infrastructure versioned as code, with a pipeline that runs on every commit?
- ☐ Is there one account per environment?
- ☐ Are the required tags applied?
- ☐ What will it cost — including the idle cost and data transfer, which are the two numbers most estimates leave out?

What gets a team sent back

In my experience it's rarely a bad idea that sinks a review. It's a **missing answer**. The usual gaps are failover, backups, alarms, the support model and cost. If your diagrams and docs cover those, you're most of the way there — [here's how I'd prepare](#).

A concession is fine — every real system has some. But write it down with a **date to fix it**. An undated exception is a permanent one.